

**UNIVERSIDAD ESTATAL PENÍNSULA DE
SANTA ELENA**

**FACULTAD DE SISTEMAS Y TELECOMUNICACIONES
CARRERA DE INGENIERIA EN SOFTWARE**



ASIGNATURA:
INTELIGENCIA DE NEGOCIOS

TEMA:
ENTREGABLE 2: ARQUITECTURA DE DATOS Y MODELO ANALÍTICO

- ELABORADO POR:**
- ANDY BRYAN ALEJANDRO VERA
 - ALISSON YAMEL REYES RICARDO
 - YANDRIS MIGUEL RIVERA TORRES

CURSO Y PARALELO:
SOFTWARE 6/1

DOCENTE:
ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD – ECUADOR

Arquitectura de Datos y Modelo Analítico

VI Inteligencia de Negocios · Ingeniería de Software · UPSE (2026-1)

1. Arquitectura de datos técnica

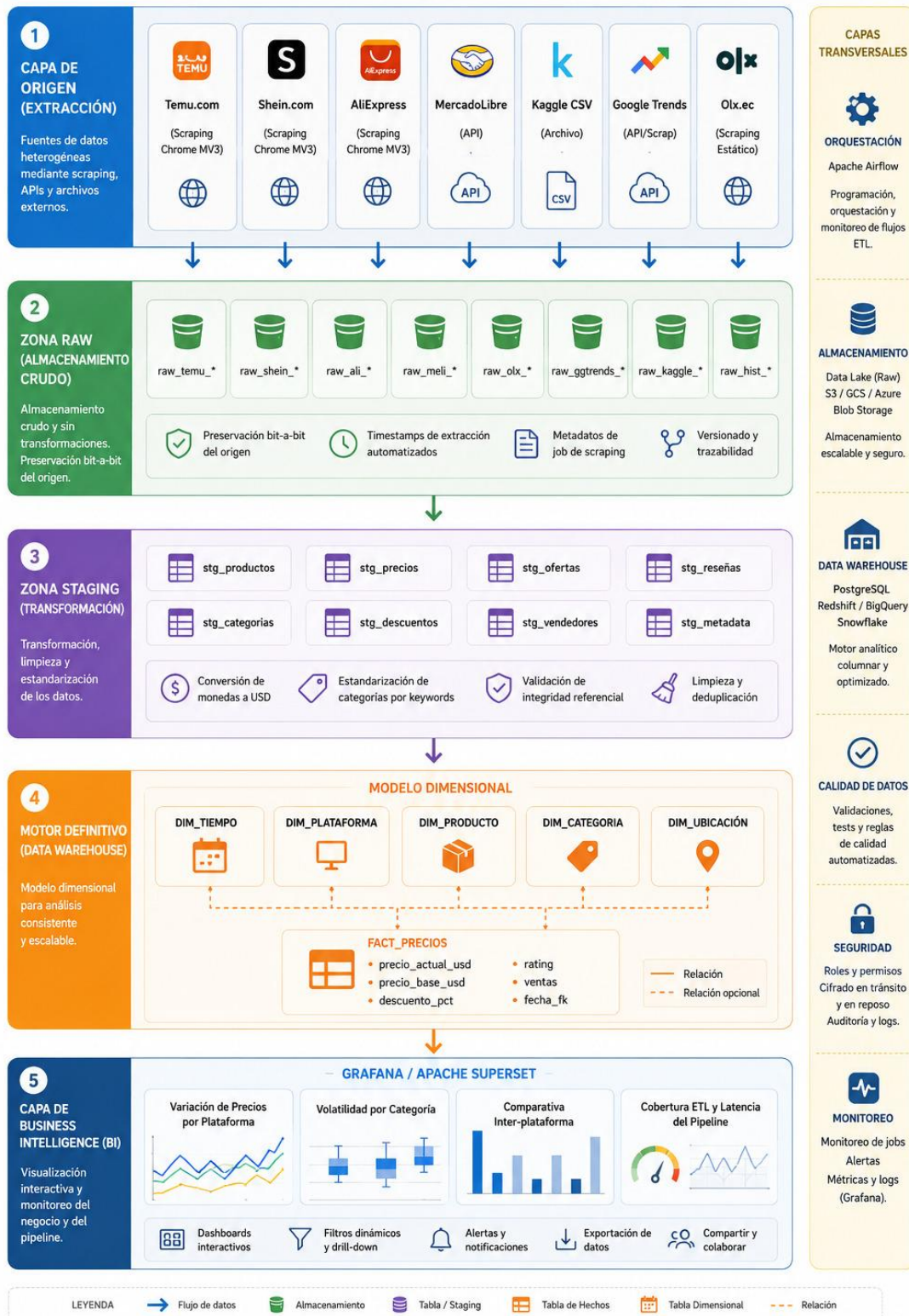
La arquitectura técnica del sistema se fundamenta en un pipeline ETL que integra múltiples fuentes de datos heterogéneas hacia un Data Warehouse centralizado. El diseño sigue el patrón de zonas progresivas de Raw a Staging al Data Warehouse, garantizando la trazabilidad completa de los datos desde su origen hasta su presentación analítica. Esta arquitectura ha sido seleccionada específicamente para abordar los desafíos de heterogeneidad inherentes al ecosistema de comercio electrónico internacional, donde coexisten datos estructurados (APIs), semiestructurados y no estructurados. La separación en zonas permite ejecutar validaciones progresivas, mantener un historial auditoría completo y facilitar la recuperación ante fallos sin pérdida de información.

El flujo de datos inicia en los orígenes externos, donde agentes especializados realizan la extracción según las características técnicas de cada fuente. Los datos crudos se depositan íntegramente en la zona Raw, preservando la información original sin modificaciones para permitir reprocesamientos completos. Posteriormente, scripts de limpieza y normalización mueven los datos hacia la zona Staging, donde se aplican reglas de calidad conversiones de tipos y estandarizaciones semánticas. Finalmente, el motor definitivo del Data Warehouse consume los datos transformados y los carga en el modelo dimensional optimizado para consultas analíticas. La capa de presentación se materializa mediante Dashboards interactivos que permiten a los analistas explorar los indicadores clave de negocios en tiempo real.

Diagrama de arquitectura técnica

ARQUITECTURA DE EXTRACCIÓN Y ANÁLISIS DE PRECIOS

DISEÑO MOCKUP - VISTA GENERAL



Tecnologías por capa

La selección tecnológica responde a criterios de escalabilidad, compatibilidad con el ecosistema open source y capacidad para procesar volúmenes de datos crecientes. PostgreSQL fue seleccionado como

motor del Data Warehouse debido a su robustez para cargas analíticas complejas, soporte nativo para tipos de datos JSON para metadatos flexibles, y capacidad de extensión mediante procedimientos almacenados en Python y PL/pgSQL. La extensión Chrome MV3 utiliza técnicas de content scripts que inyectan JavaScript directamente en el contexto de la página, permitiendo acceso al DOM completo y evitando los mecanismos de bloqueo típicos de herramientas server-side.

Capa	Componente	Tecnología	Justificación de Selección
Extracción	Scraping Browsers	Chrome Extension MV3	Acceso directo al Dom real, evadiendo restricciones CAPTCHA.
Extracción	APIs REST	Python requests + httpx	Soporte async para paralelización de llamadas.
Extracción	Archivos CSV	Pandas read_csv	Manejo eficiente de grandes volúmenes con dtype optimización.
Raw Storage	Base cruda	PostgreSQL raw_*schemas	Preservación con metadatos de auditoría.
Staging	Transformación	Python Scripts (pandas, numpy)	Flexibilidad en transformaciones complejas.
Staging	Orquestación	Apache Airflow	Programacion DAG, retry logic, logging centralizado.
DW	Motor analítico	PostgreSQL dw_*schemas	OLAP optimizado.
DW	Modelado	Data Build Tool	Versioning, testing, documentación del modelo.
BI	Visualización	Grafana + Apache Superset	Dashboards interactivos, alerts, multi-source.

BI	Reportes	Metabase	Consultas ad-hoc para analistas no técnicos.
----	----------	----------	--

2. Modelo Dimensional

El modelo dimensional constituye el núcleo analítico del Data Warehouse, priorizando la simplicidad de consulta, el rendimiento de agregación y la accesibilidad para usuarios de negocio. La tabla de hechos central captura eventos atómicos de medición de precios, mientras que las dimensiones proporcionan los contextos descriptivos necesarios para segmentar, filtrar y agregar las métricas según diferentes perspectivas analíticas.

El diseño del modelo considera específicamente la naturaleza temporal de los datos de precios, donde el mismo producto puede tener múltiples registros correspondientes a diferentes momentos de observación. Esta característica obliga a incluir la dimensión tiempo tanto en la clave primaria de la tabla de hechos como en las dimensiones descriptivas, garantizando que cada combinación producto-plataforma-tiempo sea única. Las métricas de precio, descuento y rating se almacenan como valores atómicos que permiten recalcular indicadores agregados sin pérdida de precisión, a diferencia de modelos donde se precalcular totales que limitan las posibilidades analíticas.

2.1. Declaración de Granularidad

La granularidad se define como el evento atómico de observación de precio: cada fila representa la medición del precio de un producto específico, en una plataforma específica, en un momento específico del tiempo, independientemente de si el precio cambió o permaneció igual respecto a la observación anterior.

Esta declaración de granularidad implica que, si un producto tiene tres precios diferentes observados en tres días distintos, existirán tres filas en la tabla de hechos, cada una con su propia marca temporal y valores de métrica. Este enfoque permite calcular variaciones de precio comparando filas adyacentes, detectar patrones de descuentos temporales, y analizar tendencias de precios a lo largo del tiempo sin ambigüedades. La granularidad atómica también facilita la auditoría, ya que cada dato puede rastrearse directamente a su origen y momento de extracción.

2.2. Matriz del modelo de hechos

La siguiente tabla contempla cada campo de la tabla de hechos, clasificando el tipo de dato y una descripción de su función. Donde vamos a tener los datos importantes como el precio original y el de descuento el ranking

Campo	Tipo SQL	Descripción Funcional
precios_key	bigserial	Identificador único de cada observación de precio
tiempo_key	int	Referencia temporal de la observación
plataforma_key	smallint	Plataforma de comercio electrónico origen
producto_key	Int	Producto único en el DW
Categoria_key	smallint	Categoría jerárquica del producto
ubicacion_key	smallint	Ubicación geográfica del vendedor/mercado
Precio_actual_usd	demcial(10,2)	Precio vigente al momento
Precio_base_usd	decimal(10,2)	Precio de referencia sin descuentos
Descuento_pct	decimal(10,2)	Porcentaje de descuento aplicado
rating	decimal(10,2)	Calificación del producto
num_ventas	integer	Volumen de ventas o pedidos
num_resenas	integer	Cantidad de reseñas del producto
fecha_extracción	timestamp	Momento exacto de la extracción

Estructura de las Tablas Dimensionales

Las siguientes tablas proporcionan el contexto descriptivo para segmentar y filtrar las métricas registradas en la tabla de hechos. Cada dimensión utiliza un identificador único generado por el sistema *id_[dimensión]* como clave primaria, el cual es referenciado desde *FactPrecioProducto* mediante las claves foráneas *dim_[dimension]*.

- **DimTiempo**

Campo	Tipo SQL	Descripción Funcional
id_tiempo	INT	Clave primaria identificador único del período
fecha_completa	DATE	Fecha completa de la observación (YYYY-MM-DD)
anio	SMALLINT	Año del registro (2025, 2026...)
trimestre	TINYINT	Trimestre del año (1–4)
mes	TINYINT	Mes del año (1–12)
semana_anio	TINYINT	Semana ISO dentro del año (1–53)
dia_mes	TINYINT	Día dentro del mes (1–31)
dia_semana	TINYINT	Día de la semana: 0 = Domingo ... 6 = Sábado
nombre_mes	VARCHAR(10)	Nombre del mes en español: "Enero", "Febrero"...
nombre_dia	VARCHAR(10)	Nombre del día en español: "Lunes", "Martes"...
es_fin_semana	BOOLEAN	TRUE si el día corresponde a sábado o domingo

- **DimProducto**

Campo	Tipo SQL	Descripción Funcional
id_producto	INT	Clave primaria identificador único del producto
producto_uuid	UUID	UUID de origen en la tabla Product del sistema operacional
external_id	VARCHAR(255)	Identificador del producto en la plataforma de origen
titulo	VARCHAR(500)	Nombre normalizado del producto (TRIM + LOWER aplicados)
url_producto	TEXT	URL canónica del producto en la plataforma
sku	VARCHAR(100)	SKU o código interno del producto
descripción	TEXT	Descripción del producto con etiquetas HTML eliminadas
url_imagen	TEXT	URL de la imagen principal del producto

- **DimPlataforma**

Campo	Tipo SQL	Descripción Funcional
id_plataforma	SMALLINT	Clave primaria identificador único de la plataforma
nombre_plataforma	VARCHAR(100)	Nombre legible: Temu, Shein, AliExpress, MercadoLibre...
dominio	VARCHAR(255)	Dominio web: temu.com, shein.com, aliexpress.com...
tipo_extraccion	VARCHAR(50)	Mecanismo de extracción: scraping_css, api_rest, csv_import
pais_origen	VARCHAR(100)	País de origen de la plataforma (China, Ecuador, Internacional)
activo	Boolean	TRUE si la plataforma ha sido scrapeada en los últimos 30 días

- **DimCategoria**

Campo	Tipo SQL	Descripción Funcional
id_categoria	SMALLINT	Clave primaria identificador único de la categoría
nombre_categoria	VARCHAR(100)	Categoría principal estandarizada: Ropa, Electrónica, Hogar, Calzado, Accesorios, Otro
subcategoria	VARCHAR(100)	Subcategoría específica: Camisetas, Smartphones, Muebles...
descripcion	VARCHAR(255)	Descripción textual de la categoría

- **DimUbicacion**

Campo	Tipo SQL	Descripción Funcional
id_ubicación	SMALLINT	Clave primaria surrogada de la ubicación
pais	VARCHAR(100)	País del mercado: Ecuador, China, EE.UU
region	VARCHAR(100)	Región o provincia donde opera el vendedor (si aplica)
mercado_tipo	VARCHAR(50)	Clasificación del mercado: internacional, local, regional

3. Justificación de la topología

Para este proyecto se selecciona el esquema Star Schema como la topología óptima para este caso de uso. Esta decisión se fundamenta en el balance favorable entre rendimiento de consultas, simplicidad de interpretación y facilidad de implementación, considerando que el volumen de datos proyectado.

El Star Schema elimina la complejidad de las relaciones transitivas entre dimensiones al denormalizar completamente los atributos descriptivos en tablas individuales conectadas directamente a la tabla de hechos. Esta estructura minimiza el número de joins necesarios para cualquier consulta típica de business intelligence, reduciendo el tiempo de respuesta especialmente en queries agregadas sobre métricas como precio promedio o variación porcentual. Para los KPIs, que incluyen principalmente cálculos de agregación y comparaciones temporales y entre plataformas, el Star Schema proporciona la ruta de acceso más directa a los datos requeridos.

El Star Schema es la elección óptima para este proyecto por las siguientes razones específicas alineadas con los objetivos del Entregable 1 y los KPIs definidos:

Los KPIs principales (precio promedio por categoría, tasa de variación semanal, comparativa entre plataformas) son todos cálculos agregados que se benefician directamente de la estructura desnormalizada. Queries como "precio promedio por plataforma por mes" requieren un solo scan de la tabla de hechos con agregación, sin joins intermedios.

Criterio Evaluativo	Star Schema (Estrella)	Snowflake (Copo de Nieve)	Galaxy Schema (Galaxia)
Rendimiento en Joins	Alto	Medio	Alto
Nivel de Redundancia	Alta	Baja	Media
Complejidad de Diseño	Baja	Media	Alta
Facilidad para herramientas BI	Alta	Media	Media
¿Aplica al caso?	Si	No	No

Se selecciona el Star Schema como topología del Data Warehouse por las siguientes razones:

a) Compatibilidad con herramientas BI del proyecto. Grafana y Apache Superset generan consultas SQL con JOINS directos desde una tabla de hechos hacia dimensiones desnormalizadas. El esquema estrella limita la profundidad de los joins a un único nivel, reduciendo la latencia en dashboards que deben actualizarse con datos semanales y diarios según los KPIs definidos en el E1.

b) Dominio analítico unificado. Los indicadores clave del proyecto (precio promedio por categoría, tasa de variación semanal, diferencial entre plataformas, cobertura del pipeline ETL) operan todos sobre una sola tabla de hechos central. Esta condición descarta el Galaxy Schema, cuya complejidad solo se justifica cuando existen múltiples procesos de negocio completamente independientes con tablas de hechos separadas.

c) Volumen de datos manejable. El sistema extrae datos de 8 fuentes con frecuencia semanal y diaria, produciendo miles de registros por período, no millones. A esta escala, la redundancia propia del esquema estrella en las tablas dimensionales es irrelevante en términos de almacenamiento, mientras que el beneficio en velocidad de consulta es inmediato.

d) Descarte del Snowflake Schema. Normalizar las dimensiones (separar país de región en DimUbicacion, o categoría de subcategoría en DimCategoría) agregaría JOINS adicionales sin aportar flexibilidad analítica relevante para los KPIs definidos. Las dimensiones del proyecto tienen cardinalidad baja (decenas de plataformas, categorías y ubicaciones), por lo que la normalización no generaría ahorro de almacenamiento proporcional a la complejidad introducida.

4. Diccionario de Datos

- FactPrecioProducto

Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
id_hecho	BIGSERIAL	NOT NULL, único, autogenerado		Generado automáticamente por el DW	Sistema DW
dim_tiempo	INT	> 0, FK DimTiempo	PriceHistory.capturedAt	`CAST(capturedAt AS DATE)` → lookup en DimTiempo	PostgreSQL OLTP
dim_producto	INT	> 0, FK DimProducto	Product.id	Lookup surrogado por product_uuid en DimProducto	PostgreSQL OLTP
dim_plataforma	SMALLINT	> 0, FK DimPlataforma	DomainRule.id	Lookup surrogado por domain_uuid en DimPlataforma	PostgreSQL
dim_categoria	SMALLINT	> 0, FK DimCategoría	Product.title	Clasificación por keywords del título del producto	Scraping multifuente

dim_ubicacion	SMALLINT	> 0, FK DimUbicacion	DomainRule.domain	Lookup por dominio → país en DimUbicacion	PostgreSQL OLTP
precio_actual_usd	DECIMAL(10,2)	> 0.00	PriceHistory.price	Si currency = 'USD': copia directa. Si otro: `precio * tasa_conversion`	Scraping / OLTP
precio_base_usd	DECIMAL(10,2)	> 0.00, >= precio_actual	Product.rawData.originalPrice	Extracción desde JSON rawData + conversión a USD	Scraping Temu/Shein
descuento_pct	DECIMAL(5,2)	0.00–100.00	Product.rawData.discount	`((precio_base - precio_actual) / precio_base) * 100`. Default 0 si ausente	Scraping Temu/Shein
rating	DECIMAL(3,2)	0.00–5.00, NULL si no disponible	Product.rawData.rating	CAST a DECIMAL(3,2). NULL si campo ausente en rawData	Scraping AliExpress
num_ventas	INTEGER	>= 0, NULL si no disponible	Product.rawData.sales	CAST a INTEGER. NULL si campo ausente en rawData	Scraping AliExpress
num_resenas	INTEGER	>= 0, NULL si no disponible	Product.rawData.reviewCount	CAST a INTEGER. NULL si campo ausente en rawData	Scraping multifuente
stock_disponible	INTEGER	>= 0, NULL si no expuesto	Product.rawData.stock	CAST a INTEGER. NULL si la plataforma no lo expone	Scraping Temu

- **DimTiempo**

Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
id_tiempo	INT	PK, NOT NULL		Generado por el DW	Sistema DW
fecha_completa	DATE	YYYY-MM-DD, NOT NULL	PriceHistory.capturedAt	`CAST(capturedAt AS DATE)`	PostgreSQL OLTP
anio	SMALLINT	2024–2030	capturedAt	`EXTRACT(YEAR FROM capturedAt)`	PostgreSQL OLTP
trimestre	TINYINT	1–4	capturedAt	`EXTRACT(QUARTER FROM capturedAt)`	PostgreSQL OLTP
mes	TINYINT	1–12	capturedAt	`EXTRACT(MONTH FROM capturedAt)`	PostgreSQL OLTP
semana_anio	TINYINT	1–53	capturedAt	`EXTRACT(WEEK FROM capturedAt)`	PostgreSQL OLTP
dia_mes	TINYINT	1–31	capturedAt	`EXTRACT(DAY FROM capturedAt)`	PostgreSQL OLTP
dia_semana	TINYINT	0=Dom ... 6=Sáb	capturedAt	`EXTRACT(DOW FROM capturedAt)`	PostgreSQL OLTP
nombre_mes	VARCHAR(10)	Enero ... Diciembre	capturedAt	`TO_CHAR(capturedAt, 'TMMonth')` con locale español	PostgreSQL OLTP
nombre_dia	VARCHAR(10)	Lunes ... Domingo	capturedAt	`TO_CHAR(capturedAt, 'TMDay')` con locale español	PostgreSQL OLTP
es_fin_semana	BOOLEAN	TRUE / FALSE	capturedAt	`EXTRACT(DOW FROM capturedAt) IN (0, 6)`	PostgreSQL OLTP

- **DimProducto**

Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
id_producto	INT	PK, NOT NULL		Generado por el DW	Sistema DW
product_uuid	UUID	UNIQUE, NOT NULL	Product.id	Copia directa	PostgreSQL OLTP
external_id	VARCHAR(255)	Nullable	Product.externalId	Copia directa. NULL si la fuente no provee ID externo	Scraping / API
titulo	VARCHAR(500)	NOT NULL	Product.title	`TRIM(LOWER(title))`, eliminación de caracteres especiales	Scraping / API
url_producto	TEXT	NOT NULL, formato URL válido	Product.productUrl	Validación con regex. Se descarta el registro si no es URL válida	Scraping
sku	VARCHAR(100)	Nullable	Product.sku	Copia directa. NULL si no disponible	Scraping / API
descripcion	TEXT	Nullable, sin HTML	Product.description	Eliminación de etiquetas HTML. NULL si ausente	Scraping / API
url_imagen	TEXT	Nullable, formato URL válido	Product.imageUrl	Validación con regex. NULL si no disponible	Scraping

- **DimPlataforma**

Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
id_plataforma	SMALLINT	PK, NOT NULL		Generado por el DW	Sistema DW
nombre_plataforma	VARCHAR(100)	NOT NULL	DomainRule.name	Copia directa	PostgreSQL OLTP
dominio	VARCHAR(255)	UNIQUE, NOT NULL	DomainRule.domain	`LOWER(TRIM(domain))`	PostgreSQL OLTP
tipo_extraccion	VARCHAR(50)	scraping_css \ api_rest \ csv_import	DomainRule.selectorType	`css` → `scraping_css`; `api` → `api_rest`	PostgreSQL OLTP
pais_origen	VARCHAR(100)	NOT NULL	DomainRule.domain	Lookup manual por dominio: temu.com → China, mercadolibre → Ecuador	Configuración ETL
activo	BOOLEAN	TRUE / FALSE	DomainRule.lastScrapedAt	TRUE si `NOW()` - lastScrapedAt < INTERVAL '30 days`	PostgreSQL OLTP

- **DimCategoria**

Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
id_categoria	SMALLINT	PK, NOT NULL	Sin datos.	Generado por el DW	Sistema DW

nombre_categoria	VARCHAR(100)	Ropa Electrónica Hogar Calzado Accesorios Otro	Product.title	Clasificación por keywords: "jean/camis/polo" → Ropa; "phone/celular" → Electrónica. Default: Otro	Scraping multifuentes
subcategoria	VARCHAR(100)	Nullable	Product.title	Extracción secundaria por keywords. NULL si no se identifica	Scraping multifuentes
descripcion	VARCHAR(255)	Nullable	Sin datos.	Definición manual por el equipo	Configuración ETL

- **DimUbicacion**

Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
id_ubicacion	SMALLINT	PK, NOT NULL	Sin datos	Generado por el DW	Sistema DW
pais	VARCHAR(100)	NOT NULL	DomainRule.domain	Lookup por dominio → país de mercado	Configuración ETL
region	VARCHAR(100)	Nullable	MercadoLibre API: seller.address	Extraído de la respuesta JSON de la API. NULL si no disponible	API MercadoLibre
mercado_tipo	VARCHAR(50)	internacional local regional	DomainRule.domain	Temu/Shein/AliExpress → internacional; MercadoLibre/OLX → local	Configuración ETL

5. Trazabilidad (Mapeo ETL)

Tabla DW	Campo DW	Tipo	Regla / Valores	Atributo Staging	Transformación	Fuente Origen
FactPrecio Producto	precio_act ual_usd	DECIMA L(10,2),	> 0	PriceHistory.pric e	Conversión a USD según tasa vigente al momento de captura	Scraping Temu/Shein/ AliExpress
FactPrecio Producto	precio_ba se_usd	DECIMA L(10,2)	> 0, >= actual	Product.rawData .originalPrice	Extracción de JSON rawData + conversión a USD. Default = precio_actua l si ausente	Scraping Temu/Shein
FactPrecio Producto	descuento _pct	DECIMA L(5,2)	0–100	Product.rawData .discount	((base - actual) / base) * 100. Valor 0 si no	Scraping Temu/Shein

					hay descuento	
FactPrecio Producto	rating	DECIMAL(3,2)	0.00–5.00, NULL ok	Product.rawData.rating	CAST a DECIMAL(3,2). NULL si el campo no existe en rawData	Scraping AliExpress
actPrecioProducto	num_ventas	INTEGER	>= 0, NULL ok	Product.rawData.sales	CAST a INTEGER. NULL si el campo no existe en rawData	Scraping AliExpress
FactPrecio Producto	stock_disponible	INTEGER	>= 0, NULL ok	Product.rawData.stock	CAST a INTEGER. NULL si la plataforma no expone el stock	Scraping Temu
FactPrecio Producto	dim_tiempo	INT	FK DimTiempo	PriceHistory.capturedAt	CAST(capturedAt AS DATE) → lookup id_tiempo en DimTiempo	PostgreSQL OLTP
DimProducto	titulo	VARCHAR(500)	NOT NULL	Product.title	TRIM(LOWER(title)). Eliminar caracteres especiales y espacios dobles	Scraping / API
DimProducto	external_id	VARCHAR(255)	Nullable	Product.externalId	Copia directa. NULL si la fuente no provee ID externo	Scraping / API
DimPlataforma	tipo_extraccion	VARCHAR(50)	Enum controlado	DomainRule.selectorType	'css' → scraping_css / 'api' → api_rest / 'csv' → csv_import	PostgreSQL OLTP
DimPlataforma	activo	BOOLEAN	TRUE / FALSE	DomainRule.lastScrapedAt	TRUE si NOW() - lastScrapedAt < INTERVAL '30 days'	PostgreSQL OLTP
DimCategoría	nombre_categoria	VARCHAR(100)	Lista controlada	Product.title	Keywords: "jean/camis/polo" →	Scraping multifuente

					Ropa; "phone/celular" → Electrónica; Default: Otro	
DimTiempo	es_fin_semana	BOOLEAN	TRUE / FALSE	PriceHistory.capturedAt	EXTRACT(DOW FROM capturedAt) IN (0, 6)	PostgreSQL OLTP
DimTiempo	semana_anio	TINYINT	1–53	PriceHistory.capturedAt	EXTRACT(WEEK FROM capturedAt)	PostgreSQL OLTP
DimTiempo	nombre_mes	VARCHAR(10)	Enero ... Diciembre	PriceHistory.capturedAt	TO_CHAR(capturedAt, 'TMMonth') con locale español	PostgreSQL OLTP
DimUbicacion	mercado_tipo	VARCHAR(50)	internacional local regional	DomainRule.domain	Temu/Shein/ AliExpress → internacional ; MercadoLibre/OLX → local	Configuración ETL

Mapeo de KPIs del entregable 1:

KPI (E1)	Tablas DW involucradas
Precio promedio por categoría	FactPrecioProducto.precio_actual_usd + DimCategoria.nombre_categoria
Tasa de variación semanal de precio	FactPrecioProducto.precio_actual_usd + DimTiempo.semana_anio
Diferencial de precio entre plataformas	FactPrecioProducto.precio_actual_usd + DimPlataforma.nombre_plataforma
Productos con descuento activo	FactPrecioProducto.descuento_pct > 0
Cobertura del pipeline ETL	Calculado sobre el sistema operacional: ScrapingJob.status = 'completed'